menler

CLAUDE-FIRST FELLOWSHIP · WEEK 6

SOLUTION

# Ship it — the capstone

A reference ship of StudyLoop — the Week 5 skeleton finished into a working adaptive-quiz app, with Claude on stage and a five-minute Demo Day. Yours can differ.

Week 6 · Claude · Gamma · Supabase · Replit · GitHub

*Reference build — yours can differ.*

W6

# Ship it — the capstone

How it's executed

## THE EXECUTION — HOW, WHERE & WHEN

The worked ship finishes **StudyLoop** from Week 5. The "paste notes → one question" route becomes a full **five-question adaptive round**: Claude generates the quiz, marks the answers, and re-tests whatever the student got wrong. The **visible Claude moment** is the quiz generating on screen, live, from the student's own notes. The product is **polished** so a stranger can use it on a phone, and the whole thing is pitched in **five minutes** on Demo Day.

The build happens in the Week 5 repo — no fresh start. Every working state is committed to GitHub. The deck is five slides in Gamma. The demo is rehearsed once, timed, with a recorded clip held back in case the live run stumbles. Claude is the point of the product, so it is kept on stage, not in the plumbing.

## WHAT TOOLS — AND THEIR ROLE

| | |
|---|---|
| **Claude** | The product's brain and your build partner. Generates the quiz, marks answers, and helps you write the deck and script. |
| **Your Week 5 stack** | The foundation. Lovable UI, Replit host, Supabase data — extended, not replaced. |
| **Gamma** | The deck. Five slides — Problem, Solution, Demo, Stack, Impact — from a Claude brief. |
| **GitHub** | The safety net. Each working state committed, so a bad change is one revert from safe. |
| **Replit** | The live URL. What the audience opens during the demo. |
| **A recorded clip** | The fallback. A clean run of the happy path, held back in case the live demo fails. |

THE WORKED SOLUTION

# Do it — *step by step (1 of 2)*

## 1 Build from your Week 5 scope

DO THIS

- Open the StudyLoop repo. Extend the one route into the full loop: generate 5 questions, mark answers, re-test the wrong ones.
- Work in small loops and **commit each working state** to GitHub. Do not rewrite what already runs.

**Done when:** a student can finish a five-question round end to end.

*Tip — point Claude Code at the CLAUDE.md when it drifts — the scope is still your source of truth.*

## 2 Make Claude visible

DO THIS

- Pick the one moment the audience sees the AI work: the quiz **generating live** from pasted notes.
- Show it on screen — don't pre-bake it. The visible Claude moment is what makes it a demo, not a mockup.

**Done when:** you can point at the screen and say "that is Claude, right now".

## 3 Run the polish pass

DO THIS

- Walk the five demo-killers in order and fix each: **errors** (never a blank screen), **loading** (a spinner while Claude thinks), **edge cases** (empty / huge input), **MCP reliability** (retry then fall back), **mobile** (check a narrow screen).

**Done when:** the happy path survives an empty input, a huge input, and a phone.

Safety — test with made-up notes, never a real person's private data — and ask consent before storing anything a user pastes.

THE WORKED SOLUTION

# Do it — *step by step (2 of 2)*

## 4 Build the 5-slide Gamma deck

DO THIS

- Brief Claude, then build in Gamma: **Problem · Solution · Demo · Stack · Impact** — one idea per slide.
- The deck runs two minutes. The live product is the star, so build the deck last, once the app is real.

**Done when:** five slides, each sayable in under 25 seconds.

## 5 Write the demo script & rehearse

DO THIS

- Ask Claude to draft the arc: problem → solution → live demo → impact. Trim it to fit five minutes.
- Rehearse once out loud, time it, and **record a fallback clip** of the happy path.

```
DEMO SCRIPT — SHAPE
0:00  Problem      — students re-read, don't self-test
0:30  Solution     — paste notes, get an adaptive quiz
1:00  Live demo    — paste notes -> Claude builds the quiz
3:30  Stack+Impact — how it's built, who it helps
4:30  Close + ask  — what I'd build next
```

**Done when:** you can deliver the whole thing in five minutes without the slides.

## 6 Demo live & submit the write-up

DO THIS

- Present: two minutes of deck, three minutes live, Claude on screen. Answer Q&A honestly — "here's what I'd do next" is a strong answer.
- Submit the **README**, the **demo video**, and a paragraph on what you built and learned.

**Done when:** you have demoed it live and handed in the proof.

YOUR OUTPUT — THE 5-SLIDE DEMO DAY DECK

```
1 · PROBLEM    Students re-read notes instead of
               testing themselves — and forget fast.
2 · SOLUTION   StudyLoop: paste notes, get an adaptive
               quiz that re-tests what you got wrong.
3 · DEMO       Live — paste notes, Claude builds the
               quiz on screen, marks it, adapts.
4 · STACK      Lovable + Replit + Supabase + Claude.
               Built solo in one week.
5 · IMPACT     Active recall, on any notes, in seconds.
               Next: spaced repetition + classes.
```

TOOL OVERVIEW

# What you'll find *where*

Every surface this ship uses, and the one job it does — build, present, prove.

| | |
|---|---|
| **Claude Code** | Your editor. Where the Week 5 repo becomes a finished product, debugged with Claude. |
| **CLAUDE.md** | Repo root. Still the source of truth — point Claude back at it when it drifts mid-build. |
| **GitHub repo** | Your repository. Commit each working state; a bad change is one revert away. |
| **Replit** | Your deploy. The live URL the audience opens during the demo. |
| **Gamma** | Your deck workspace. Five slides from a Claude brief. |
| **The recording** | A fallback clip of the happy path, ready if the live run stumbles. |

MAKE IT REUSABLE

# Make it *run on its own*

A shipped product feels alive when parts of it keep working without you. These three surfaces turn a demo into something that runs.

## CONNECT A CONNECTOR

**Where:** Customize → Connectors → + → pick the app → sign in → Allow. Switch it on in the chat.

1. Connect the app that holds real user data — a calendar, a notes app, a sheet.
2. Sign in and Allow — Claude never sees the password.
3. Switch it on so the product can read live data, not samples.

**For this project:** StudyLoop could pull a student's real class notes from a connected notes app instead of a paste box.

*Give it the narrowest scope that works — one folder, one calendar.*

## MAKE A ROUTINE

**Where:** Left strip → the Routine you build. A saved multi-step job the product can trigger.

1. Chain the steps of one feature: read wrong answers → draft a revision set → save it.
2. Name each step so the flow is readable.
3. Trigger it from the app — or let it run on its own.

**For this project:** "Build tomorrow's revision set from today's mistakes" is one Routine the product runs for every student.

## MAKE A SCHEDULE

**Where:** Left strip → Scheduled → New task → describe the job → pick the rhythm → Schedule.

1. Describe it: "every evening, email each student their three weakest topics."
2. Pick **daily**.
3. Pause or delete from the same screen.

**For this project:** the nightly nudge makes StudyLoop feel alive between sessions. **When to run:** on a rhythm. **When to stop:** Scheduled → Pause.

PROMPT LIBRARY · STAR

# Write it in *STAR*

Four levels of the ship-and-show prompt — Situation, Task, Action, Result. Start basic, grow into pro.

## BASIC

- S **Situation.** I have a skeleton from last week and need it finished.
- T **Task.** Help me extend one route into a working feature.
- A **Action.** Read my CLAUDE.md and repo, then build the full five-question loop in small, committed steps.
- R **Result.** One flow that works end to end, on a live URL.

## INTERMEDIATE

- S **Situation.** My app works but it breaks in a live demo.
- T **Task.** Run a polish pass on the happy path.
- A **Action.** Add error messages, loading states, edge-case handling, an MCP retry, and a mobile check.
- R **Result.** A product a stranger can use on a phone without it stalling.

## ADVANCED

- S **Situation.** I need a five-minute pitch and a deck, not just an app.
- T **Task.** Draft the deck and the demo script.
- A **Action.** Build five slides — Problem, Solution, Demo, Stack, Impact — and a timed 2-min-deck / 3-min-live script.
- R **Result.** A deck and a script I can deliver in five minutes, timed.

## PRO

- S **Situation.** I want the product to feel alive, not static, on Demo Day.
- T **Task.** Add one autonomous feature the audience can see.
- A **Action.** Wire a Routine or Schedule — a nightly revision set from the day's wrong answers — and show it firing live.
- R **Result.** A product that keeps working between sessions, with Claude visibly in the loop.

WHY IT MATTERS

# Where this pays off *in real life*

A shipped, demoed product is the most convincing thing you own. It is proof, not a promise.

- **A portfolio piece** — a live link and a demo video beat any CV line about "familiar with AI".
- **A startup MVP** — the capstone is often the first version of something real, built in a week.
- **A pitch** — the five-minute Demo Day arc is the same shape you use to pitch an idea to anyone.
- **A CV project** — "shipped and demoed an AI product solo" is a sentence that gets interviews.

WHAT GREAT LOOKS LIKE

- The product **works end to end** — one flow, done well, not five half-built.
- Claude is **visible** — the audience can see the AI do its job live.
- The happy path **survives** an empty input, a huge input, and a phone.
- You can deliver the **five minutes** under pressure, with a fallback ready.